

CA02 Viva – Mock Questions & Model Answers

MediConnect: Microservices-Based Clinic Appointment Booking Platform

Practice tip: answer aloud using Driver → Decision → Alternative → Trade-off, and finish every answer with a trade-off.

A. System overview

Q1. In one minute, describe your system.

MediConnect is a microservices web platform where patients book clinic appointments in advance and then attend the clinic in person. A React single-page app talks to an API Gateway, which routes requests to four independent services (Auth, Appointment, Records, Notification). Each service owns its own database. Services communicate synchronously over REST through the gateway and asynchronously through a RabbitMQ event bus, and everything is containerised and deployed on Kubernetes.

Q2. Why did you choose this problem, and is it a real-world problem?

Clinic appointments are still often handled by phone or walk-in, which causes long queues, double-booking, and no visibility of doctor availability. MediConnect lets patients see availability and reserve a slot in advance, reducing waiting while keeping the consultation in person. It is a genuine, common problem, and it is rich enough to demonstrate real architectural concerns such as scalability, availability, and security.

B. Architecture style

Q3. Why did you choose a microservices architecture?

The driver was independent scalability (booking load can spike), fault isolation, and maintainability as the platform grows to many clinics. We considered a monolith, which is simpler but scales as a single block and one failure can bring the whole system down, and a plain client-server style, which does not isolate faults or scale functions independently. The trade-off we accepted with microservices is higher operational complexity: orchestration, distributed data, and eventual consistency.

Q4. Isn't microservices overkill for a clinic booking system?

For a single small clinic, yes – a modular monolith would be a reasonable and simpler choice. We chose microservices to meet the scalability and availability goals as the platform grows across many clinics, and to isolate failures such as payment from booking. It is a deliberate trade-off: we accept more operational complexity in exchange for independent scaling and fault isolation.

Q5. Why do you need an API Gateway?

It gives a single public entry point and centralises cross-cutting concerns: authentication (JWT verification), TLS termination, routing, and rate limiting, while hiding the internal services from clients. The alternative – clients calling services directly – exposes every service, duplicates authentication, and creates CORS problems. The trade-off is that the gateway is a potential single point of failure, which we mitigate by running multiple gateway replicas behind a load balancer.

C. Communication between components

Q6. How do your services communicate with each other?

Two ways. Synchronously over REST through the gateway for request/response flows (for example, booking), plus one direct service-to-service REST call where the Records Service validates an appointment with the Appointment Service. Asynchronously through RabbitMQ for events: when an appointment is booked, the Appointment Service publishes an `appointment.booked` event that the Notification and Records services consume independently.

Q7. Why use asynchronous messaging instead of just REST calls everywhere?

So the booking response does not have to wait for notifications or record creation, which keeps the system responsive, and so a slow or failed downstream service does not cascade into the booking path. If we used synchronous calls, a down Notification Service would fail the booking. The trade-off is eventual consistency and a message broker to run and monitor.

Q8. Why RabbitMQ and not Kafka?

RabbitMQ is simpler to operate and is a great fit for task and event queues at our scale. Kafka is designed for very high-throughput event streaming and log/analytics workloads, which we do not need yet. Choosing RabbitMQ is a right-sizing decision; if we later needed high-volume event streaming, Kafka would become justified.

D. Data

Q9. Why database-per-service instead of one shared database?

Database-per-service keeps services loosely coupled: each service owns and can evolve its schema and scale its store independently. A shared database creates hidden coupling – one team's schema change can break another service, and it becomes a scaling bottleneck. The trade-off is that cross-service queries are harder and we need the Saga pattern for consistency across services.

Q10. Healthcare needs strong consistency (ACID) – why did you list MongoDB?

We use polyglot persistence: PostgreSQL for transactional data such as bookings and payments, where ACID guarantees matter, and MongoDB for flexible, document-shaped data such as consultation records and prescriptions, whose structure varies. To be transparent, the CA03 prototype simplified this to MongoDB only to keep the stack light; the design keeps PostgreSQL for the transactional core.

Q11. How do you keep data consistent when it is spread across services?

For critical multi-step operations such as booking plus payment, we use the Saga pattern: a sequence of local transactions with compensating actions that undo earlier steps if a later one fails. Events carry state between services, and consumers are idempotent so a repeated event does no harm. The trade-off versus a two-phase commit is that consistency becomes eventual, but we avoid the blocking and tight coupling of distributed transactions.

E. Security

Q12. How is patient data protected?

Defence in depth: TLS 1.2+ for all traffic in transit, encryption at rest for records and payment data, JWT with role-based access control enforced at the gateway, input validation in each service, and audit logging for compliance. Payment is isolated and never stores raw card data, delegating to a compliant payment provider.

Q13. Why JWT and OAuth 2.0 instead of server-side sessions?

JWT is stateless: the token carries the user's identity and role, so any replica of any service can verify it without a shared session store, which is what lets us scale horizontally. Roles in the token drive access control (patient, doctor, admin). Server-side sessions would need sticky sessions or a shared session store. The trade-off with JWT is that revoking a token before it expires is harder, so we use short token lifetimes.

Q14. How does the gateway enforce security for the other services?

The gateway verifies the JWT on every protected route and only then forwards the request, injecting the authenticated identity (user id and role) as trusted headers. The internal services are not exposed to the outside network, so they can trust those headers. The gateway also applies rate limiting to reduce abuse.

F. Scalability, availability, performance

Q15. How would the system handle 10,000 concurrent users?

The services are stateless behind the gateway, so Kubernetes' Horizontal Pod Autoscaler replicates the busy service – typically the Appointment (booking) service – independently of the lighter ones. A load-balanced, replicated gateway spreads incoming traffic, Redis caches hot reads such as doctor listings and slot availability, and database indexing keeps search and booking queries fast.

Q16. What happens if one service crashes – does the whole system go down?

No. Service boundaries contain failures. If, say, the Notification Service crashes, patients can still book and doctors can still record visits; the `appointment.booked` events queue durably in RabbitMQ and are processed when the service recovers. Kubernetes self-healing restarts the failed service automatically. This is the core of our 99.9% availability goal.

Q17. How do you meet your performance target?

The 95th-percentile target is under 500 ms. The gateway routes efficiently and offloads TLS, Redis caches frequently read data so we avoid repeated database hits, and we index the doctor, clinic, and slot fields so search and booking queries stay fast. Because responses are lightweight JSON, payloads are small.

G. Diagrams and design decisions

Q18. Walk me through what happens when a patient books an appointment.

The browser sends the request to the gateway, which verifies the JWT and forwards it to the Appointment Service. That service checks the slot is free, reserves it, records a (simulated) payment, saves the appointment, and publishes `appointment.booked`. The Notification Service consumes the event and creates a confirmation; the Records Service consumes the same event and creates a pending consultation record for the doctor to complete after the visit.

Q19. Explain your component diagram – how is a single service structured?

Each service is layered: a REST controller exposes the API endpoints, a business-logic layer performs validation and rules, and a data-access (repository) layer talks to the database, with an event publisher/consumer for messaging. Upper layers never bypass lower ones – for example, the controller never touches the database directly – which keeps the service maintainable.

H. Critical / curveball

Q20. What are the weaknesses or risks of your architecture?

The main costs are operational complexity (many services, orchestration, monitoring), eventual consistency in the event-driven paths, and two components – the gateway and the broker – that must be made highly available through replication and clustering. Debugging a request that spans several services is also harder, which is why centralised logging and tracing are part of the design.

Q21. If you rebuilt this from scratch, what would you change?

For an early-stage single-clinic deployment I would likely start with a modular monolith and extract microservices only where scaling or team boundaries demand it – following the “everything is a trade-off” principle. I would also add distributed tracing from day one to make cross-service debugging easier, and firm up the Saga compensations for the booking-payment flow.